LiveFire Fusion PoC Implementation Plan

Oct 20, 2026

Summary and decisions

We build one complete LiveFire loop on the core Microsoft stack in Python:

1. A bounded Key Vault fault breaks ordering.
2. An independent customer journey catches the False Green.
3. Two governed agents produce a tested repair.
4. The same journey verifies the order.

The whole system rebuilds from one command in each time-bound CloudLabs lab. Target: ready for the walkthrough by Nov 18, 2026.

Decision	Choice	Reason
Where it runs	CloudLabs guided lab: fixed duration, deallocated automatically when it ends, custom RBAC and policy (CloudLabs guide)	Provided for the hackathon; it is why we need the one-command rebuild in the lab section
How it is shown	Live customer walkthrough, probably 45 minutes plus Q&A	Supervised mode lets the presenter pause at each gate and explain
Stack	Core Microsoft stack, simplest options	Same story and controls with fewer moving parts (see Scope)
Language	Python 3.12 for the order service, MCP servers, workflow, referee and Control Center	One toolchain; Agent Framework, the MCP SDK and Playwright all support Python
Model	One GPT-5.x deployment shared by both agents	gpt-4o (2024-11-20) retired on Oct 1, 2026 (source); the day-1 probe confirms which model and quota the lab region offers

Decision	Choice	Reason
Autonomy	One policy object: Supervised by default, Autonomous Twin as a switch, Advisory stops before any action	Three modes for the cost of one
Approval	Button in the Fusion Control Center that records the approver and the decision latency	Teams cards need tenant admin work, so they move to stretch scope
Source and CI	A GitHub repo the team controls, GitHub Actions, images in GitHub Container Registry (GHCR)	Survives lab resets; the org is still to be confirmed

The lab's permissions are unknown until we test them, so the plan has two paths. With **Path A** (role and policy assignments allowed), we build the blueprint scenario as written. With **Path B** (Contributor only), the story stays the same but the fault moves to a Key Vault access policy. The day-1 probe decides which path applies.

Scope

Everything that makes LiveFire different stays real. The time savings come from hosting, packaging and presentation polish.

Stays real

1. **Independent referee.** A pinned Playwright order journey, plus a read-only SQL check that exactly one order and one payment authorization were saved. No model decides pass or fail.
2. **Real fault.** A bounded, reversible fault inside the twin, then a narrow restore.
3. **Platform refusal.** The platform refuses the over-broad recovery option, live.
4. **Separated agents.** Two agents with separate tool catalogs and credentials. All tool access goes through MCP, and one cross-role denial is shown.
5. **Agent-written fix.** A candidate on a branch: patch, regression test and runbook diff. CI builds and scans it, it deploys as a new twin revision, and the unchanged journey verifies it.
6. **Evidence and scorecard.** A deterministic verdict and one evidence package per drill ID. The business scorecard shows verified recovery time, failed orders, backlog, decision latency, tokens and estimated cost.

7. **Safety stops.** Kill switch and reset.

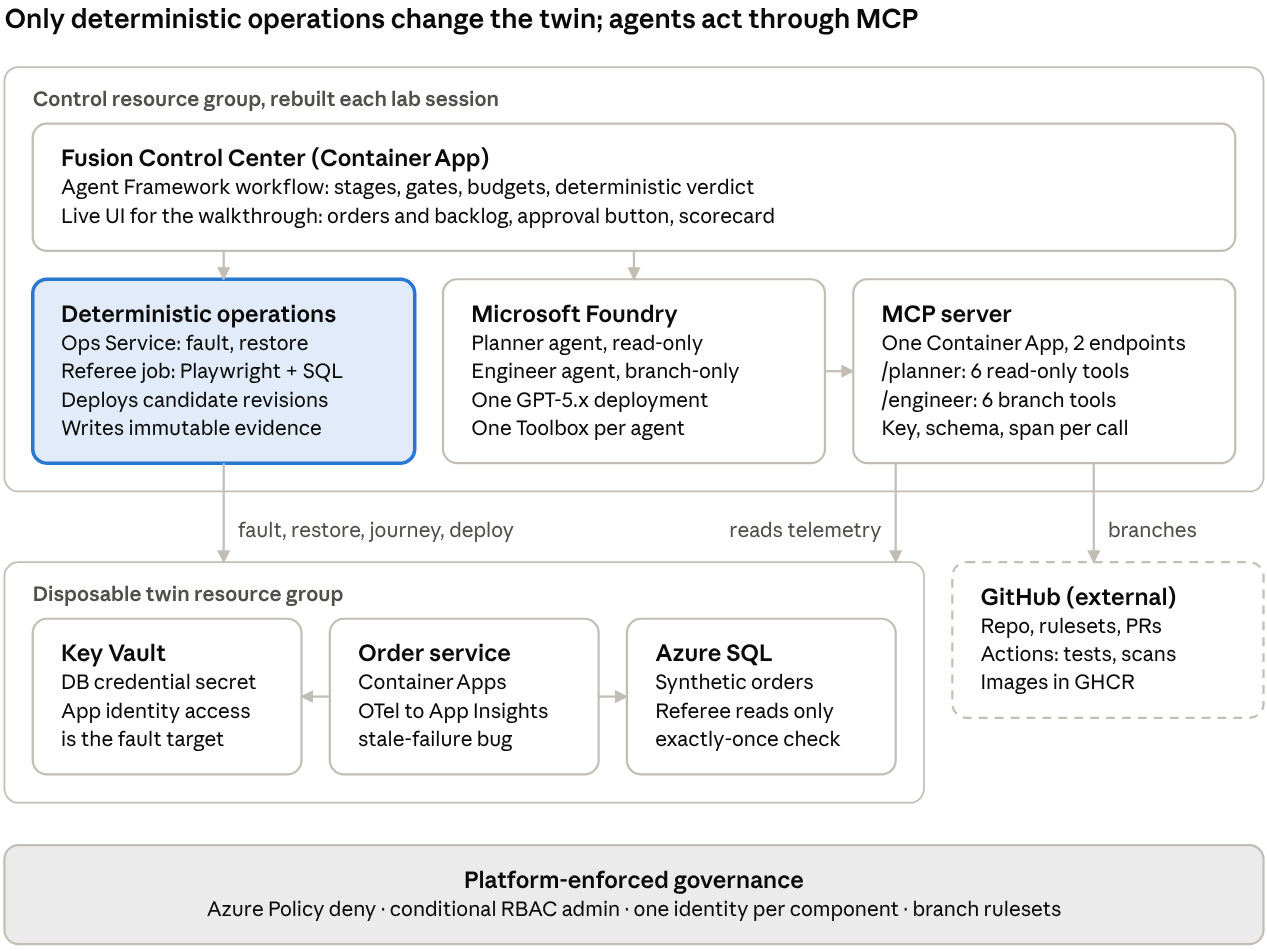
Simplified

Blueprint item	PoC version	What it saves
Twin rebuilt from Bicep for every drill	Bicep deploys the twin once per lab session. A reset script restores access, rolls back to the v1 revision and clears drill orders between runs	A full redeploy per drill
Foundry Hosted Agents	Foundry Agent Service prompt agents, defined in YAML and created by script	No agent containers to build and deploy
Two MCP deployments	One Container App with two endpoints, each with its own key and tool list	One service to run (the blueprint allows this)
Functions fault injector	Ops Service: an internal Container App with its own identity and a fixed list of allowed actions	No Functions plan; same environment as the rest
Playwright Workspaces	Container Apps Job running the pinned journey in the official Playwright image	No extra service; Workspaces can be added later
Teams Adaptive Card	Approval button in the Control Center	No Teams app or admin consent
GitHub Advanced Security and image signing	CodeQL (Bandit if the repo must stay private), gitleaks, pip-audit and Trivy in Actions; image digest recorded	Free on a public repo; signing moves to the roadmap
Immutable evidence store	Blob versioning with version-level immutability and a short, unlocked retention period	A locked policy could block lab teardown
Metered drill cost	Token counts from traces times the price sheet, plus a fixed per-drill infrastructure estimate	No billing export needed

Blueprint item	PoC version	What it saves
Three autonomy modes	One policy object with a mode field	One code path

Cut to roadmap: Chaos Studio, Teams cards (stretch, if time allows), Power BI, Power Apps, AKS, Logic Apps, App Service, Azure Recovery Services, signing and provenance, additional scenarios or applications, fleet learning.

Target architecture



LiveFire PoC architecture · control plane, disposable twin, GitHub

Agents reach the world only through MCP, and every change to the twin goes through the deterministic operations. Platform governance sits under everything. The control and twin resource groups each get their own Container Apps environment, so tearing down the twin never touches the control plane.

Component	Azure service	Why this choice
Control Center and workflow	Azure Container Apps running Microsoft Agent Framework (Python)	UI, workflow and approval gate in one container; checkpoints in Blob storage
Planner and Engineer	Microsoft Foundry Agent Service prompt agents, defined as code	Generally available; Foundry traces show every tool call; no agent containers to build
Model	One GPT-5.x deployment (Global Standard)	gpt-4o is retired; one deployment serves both agents
Agent tools	Foundry Toolboxes connecting to an MCP server on Container Apps (Python MCP SDK)	Toolboxes are generally available and keep credentials out of agent code (source)
Evidence search	Azure AI Search, Basic or Free tier	About 15 curated documents: incidents, runbooks, architecture notes
Fault, restore, reset, deploy	Ops Service: an internal-only Container App with its own managed identity	A fixed list of allowed actions; replaces the Functions injector
Referee	Container Apps Job: the pinned Playwright journey plus a read-only SQL check	Deterministic, with its own identity; Playwright Workspaces optional
Twin	Container Apps, Azure SQL (serverless), Key Vault, App Insights and Log Analytics, all from Bicep	Revisions make candidate deploys and rollbacks a single call
Source, CI, images	GitHub repo, GitHub Actions, GHCR	Survives lab resets; CodeQL is free on a public repo
Evidence store	Blob Storage with versioning and version-level immutability	Short, unlocked retention, so lab teardown still works
Telemetry	Azure Monitor OpenTelemetry distro	One drill ID across the app, workflow, MCP and agents
Governance	Azure Policy, Azure RBAC with conditions, managed identities	The platform refuses unsafe actions, not the model

Working inside a time-bound CloudLabs lab

Treat every lab launch as disposable. One command brings up the whole system in a fresh lab, and nothing important lives only in the lab.

- **One command.** `azd up` (or `./bootstrap.sh`) runs Bicep for both resource groups, then post-provision scripts. The scripts seed SQL, load the AI Search index, and create the model deployment, both agents and their Toolboxes from YAML. They also set up the GitHub OIDC credential and run the baseline journey. Target: under 30 minutes, measured in week 1.
- **Agents as code.** Agent instructions, tool lists and output schemas live in the repo. Nobody builds agents by hand in the portal.
- **State outside the lab.** Code, runbooks, the incident corpus and container images live in GitHub and GHCR. Each lab only holds a running copy.
- **Develop off the lab.** Each engineer runs the order service, SQL Server and a fake secret source locally in Docker Compose. MCP tools and the workflow run against recorded fixtures. The lab is for integration runs, so five people don't queue for one environment.
- **One lab per engineer** if CloudLabs allows it, each bootstrapped with the same command.

Day-1 probe. Run `lab-probe.sh` (shared with this plan) in the first lab session. It creates throwaway resources and records which of these the lab allows:

- Role assignments, plain and with conditions
- Policy definitions and assignments
- Entra app registrations
- Both Key Vault permission models
- Container Apps, Azure SQL (SQL and Entra-only authentication), AI Search and immutable Blob storage
- GPT-5.x deployments and quota
- Federated credentials
- Outbound access to GitHub

It then deletes everything it created. It has not been run in a live lab yet, so expect small fixes the first time.

Probe result	Path	Fault	Refusal of the broad option	Automation identities
Role and policy assignments allowed	A (blueprint)	Remove and restore the app identity's Key Vault Secrets User role on the twin vault	An Azure Policy deny on Owner, Contributor and User Access Administrator assignments. In addition, the recovery identity holds Role Based Access Control Administrator with a condition that allows only Key Vault Secrets User (Microsoft Learn)	One managed identity per component, each with scoped roles
Contributor only	B (fallback)	Vault in access-policy mode: remove and re-add the app identity's get/list secret policy, which Contributor can change	Azure Resource Manager rejects the role assignment for lack of permission (AuthorizationFailed). Azure Policy is used only if the organizers assign it in advance	The lab-provided service principal, if the Environment tab offers one

Action on day 1, whatever the probe says: ask the hackathon organizers or CloudLabs support (cloudlabs-support@spektrasystems.com) for Owner on the lab resource group, and for the lab duration per launch. Getting Path A is worth one email.

Scenario engineering

The False Green is a deliberate, deterministic bug in the app. After access returns, the order path keeps a stale failure state. The dependency health check makes a fresh call, so it turns green.

Order service (FastAPI on Container Apps)

- **Endpoints:** a shop page, `POST /orders`, `GET /orders/{id}`, `/health` (process is up) and `/health/dependencies` (a fresh Key Vault read plus a SQL ping).

- **Order path:** each order reads the database credential through a `SecretProvider` with a short cache of about 30 seconds, so secret rotation still works. It then writes the order and a synthetic payment authorization to SQL and returns a confirmation number.
- **Seeded weakness:** when a refresh fails with 403, the provider marks the secret unavailable and never retries. It arrives in a commit along the lines of "cache secret failures to stop hammering Key Vault", which gives the Planner some history to find.
- **Telemetry:** OpenTelemetry through the Azure Monitor distro. That covers `orders_completed` and `orders_failed` metrics, dependency spans, and the drill ID on every span.
- **Load generator:** a Container Apps Job places a steady trickle of synthetic orders and keeps failed ones in a backlog that it retries. Throughput, backlog and the drain after recovery are therefore real numbers.

Fault, restore and timing

- **Path A fault:** delete the app identity's Key Vault Secrets User assignment on the twin vault. **Restore:** recreate exactly that assignment.
- **Path B:** the same change, made through the vault's access policy.
- **Propagation:** Azure Resource Manager caches role data. Microsoft says role assignment changes can sometimes take up to 30 minutes to take effect (Microsoft Q&A).
  - The workflow never waits on a fixed timer. It polls the dependency probe until it sees 403 (fault active) or 200 (restored), shows a "waiting for access change" state in the UI, and records the measured delay.
  - Measure both paths in week 1. If the delay is too long for the walkthrough, add a Key Vault network-rule fault behind the same Ops Service interface, and keep role removal as the narrated scenario.

Expected fix from the Engineer

- **Patch** to `secret_provider.py`. On 401, 403 or a timeout, it retries with bounded exponential backoff and a half-open check, recreates the client once access returns, and never caches a failure for longer than the refresh window.
- **Regression test** `tests/test_secret_recovery.py`. A fake secret source fails with 403 and then succeeds. The test asserts that the provider recovers within the backoff budget and that an order then succeeds.

- **Runbook diff** to `runbooks/order-service-keyvault-recovery.md`. Version 1 stops at "confirm /health is green". The update adds: run the customer-journey check; if it fails after access is restored, restart the revision as a safe fallback and raise a defect.

Evidence the agents can find (synthetic and curated)

- Three to five incident records in AI Search. One is an earlier "orders failed after secret rotation, fixed by restart", which is the recurring operational debt. Another contains a planted prompt-injection line for test T-08.
- The architecture note, runbook v1 and commit history in the repo.
- The referee's failed journey: the failing step, the error class, trace IDs, and zero saved orders for the drill ID.

Golden journey and check

- **Journey:** Playwright for Python, one browser, stable test IDs. It opens the shop, adds a product, checks out with the drill ID, and reads the confirmation number.
- **SQL check:** a read-only login confirms exactly one order and one payment authorization for that drill ID.
- **Protection:** the journey lives under `journeys/`. It is protected by a branch ruleset and CODEOWNERS, and it is on the Engineer endpoint's deny list.

Agents, MCP tools and workflow

Both agents are Foundry Agent Service prompt agents, and each reaches the outside world only through its own MCP endpoint. An Agent Framework workflow in Python runs every stage, gate and budget around them.

Agents (defined in `agents/*.yaml` and created by the bootstrap)

Agent	Sees	Produces	Cannot
Investigation and Recovery Planner	Its Toolbox, which reaches the `/planner` endpoint; the drill objective and referee results from the workflow	`RecoveryOptions` on the first pass; a `RemediationBrief` after a False Green	Write anything, run recovery, deploy, or mark recovery complete

Agent	Sees	Produces	Cannot
Remediation Engineer	Its Toolbox, which reaches the `/engineer` endpoint; the validated brief and its allowed paths	`ChangeReport`: branch, commits, changed paths, and a map from evidence to each change	Read raw telemetry, touch `journeys/`, `.github/`, `infra/` or `main`, merge, or deploy

Both agents return JSON. The workflow validates it against Pydantic models in a shared `contracts` package. The contracts are frozen at gate G1 (Oct 26).

MCP tool catalogs (Python MCP SDK, streamable HTTP, one Container App with two endpoints)

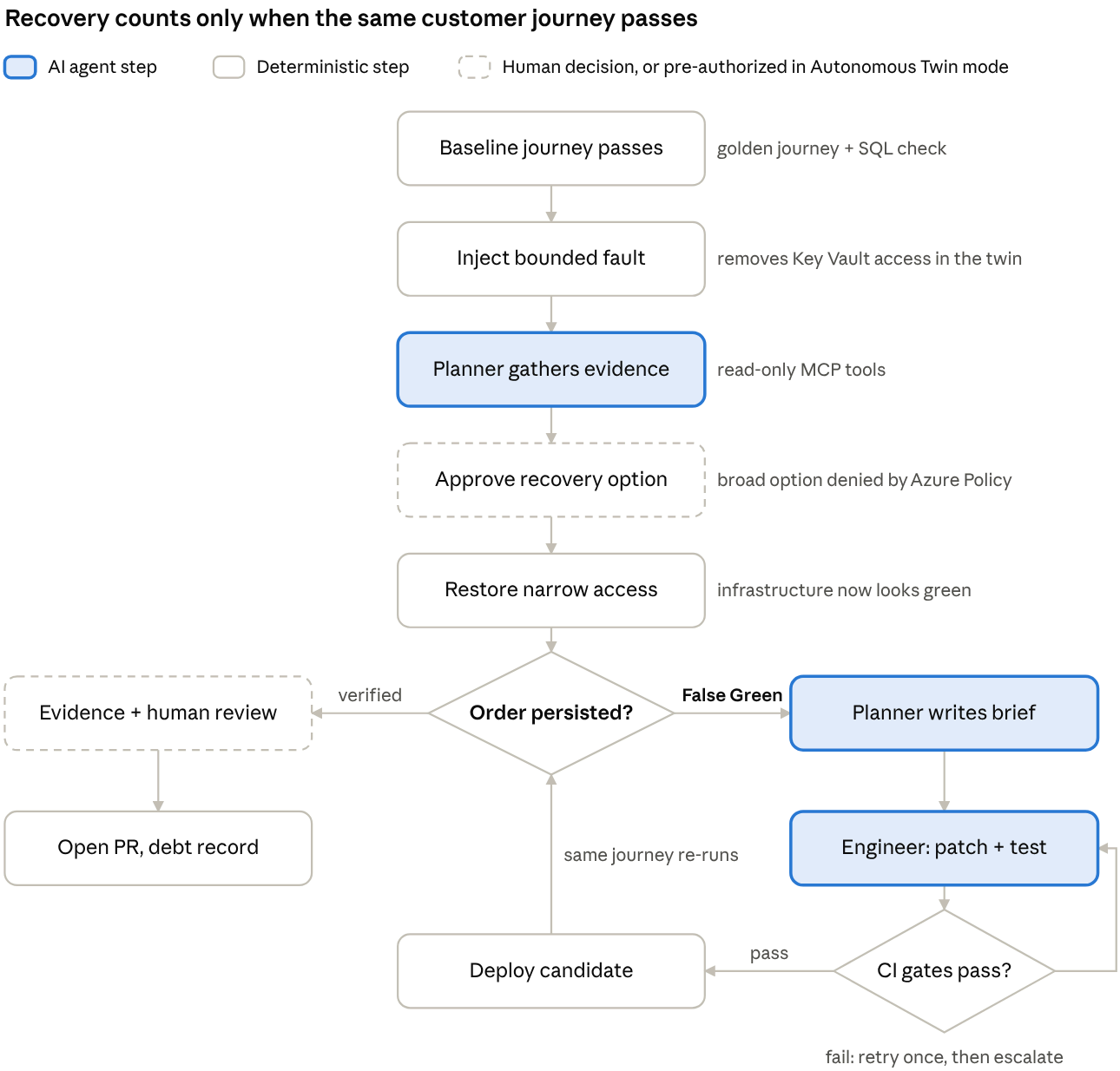
Endpoint	Tool	Limits
`/planner`	`query_incident_telemetry`	Fixed KQL templates per signal (requests, exceptions, dependencies, metrics), filtered by drill ID and time window; no free-form KQL
`/planner`	`get_identity_snapshot`	Twin vault only: role assignments or access policies for the app identity; never tokens or secret values
`/planner`	`search_incident_history`	Filtered by service, top 5 results
`/planner`	`read_runbook`	Runbook folder at the pinned commit
`/planner`	`read_repository_context`	Allowlisted paths, with a size cap
`/planner`	`get_commit_history`	One path, last 10 commits
`/engineer`	`read_allowed_files`	Only the paths named in the brief
`/engineer`	`create_candidate_branch`	Creates `candidate/<drill-id>-<attempt>` from the pinned base commit
`/engineer`	`apply_patch`	Allowed paths only; at most 3 files and 200 changed lines
`/engineer`	`write_regression_test`	`tests/` only
`/engineer`	`update_runbook`	`runbooks/` only

Endpoint	Tool	Limits
`/engineer`	`get_ci_results`	Candidate branch only: job status and the first error lines

Server-side checks. Every call is checked on the server: endpoint key, JSON schema, and path and size limits. Each call also emits a span with the drill ID, caller, tool, argument hash and the allow or deny decision.

Credentials. Only the MCP server holds GitHub credentials, which it reads from Key Vault. `/planner` uses a read-only token. `/engineer` uses a token that a ruleset limits to `candidate/*` branches.

Ingress. Foundry calls the endpoints from Microsoft's network, so the MCP app needs public ingress with key authentication.



Drill workflow · one False Green loop, at most two candidates

The referee's verdict is the only exit. A False Green sends the drill into the repair loop, and every candidate is checked by the unchanged journey before anyone reviews it.

Workflow rules

- **Deterministic steps.** Baseline, fault, policy check, restore, CI gates, deploy, referee and packaging are plain code in the workflow, never agent tools.
- **Checkpoints.** Each stage writes a checkpoint to Blob storage, so a run can resume after a container restart.

- **Budgets.** Two candidate attempts, a token cap and a tool-call cap per agent, and a timeout per stage. Any breach ends in a named escalation state, such as `ESCALATED_BUDGET`.
- **Verdict.** Recovery is declared only when this expression is true, and no model can set it:

```
verified_recovery = authorized_action_completed AND policy_passed
                    AND technical_health_passed AND ci_gates_passed
                    AND journey_passed AND persisted_order_assertion_passed
```

Governance and safety controls

Every control the walkthrough claims is enforced outside the model. The PoC softens only retention locking, image signing and, where the lab requires it, sign-in on the approval gate.

Control	Enforced by	PoC status
Twin isolation	Separate twin resource group with no peering or route to anything else; operations identities are scoped to it	Real
Broad permission refused	Azure Policy deny (Path A) or Azure Resource Manager authorization (Path B)	Real
Recovery limited to one role	Role Based Access Control Administrator, with a condition that allows only Key Vault Secrets User (Path A)	Real
Agents hold no Azure or GitHub credentials	Agents only have Toolbox connections to MCP; the MCP server holds the credentials	Real
Each agent limited to its own tools	Separate endpoint keys and tool catalogs; a cross-role call returns 403 and is logged	Real

Control	Enforced by	PoC status
Agents cannot change the acceptance test	Branch ruleset and CODEOWNERS on `journeys/`, the Engineer endpoint's deny list, and the referee always runs the pinned commit	Real
Recovery decided by evidence	Deterministic verdict in workflow code	Real
Human authority	Approval gate; the pull request is left unmerged; no production identity exists	Real
Approver identity	Entra sign-in (Easy Auth) on the Control Center if app registrations are allowed; otherwise a presenter passcode	Real, or simplified
Evidence integrity	Blob versioning and version-level immutability with a short, unlocked retention period	Simplified (locked in production)
Supply chain	Tests, CodeQL or Bandit, gitleaks, pip-audit and Trivy; image digest recorded	Simplified (signing on the roadmap)
Prompt injection	Tool output is labelled as data, tools cannot act, and test T-08 plants an instruction	Real (one scenario)
Runaway autonomy	Attempt, token, tool-call and time budgets; the kill switch reverses the fault	Real
Secrets kept out of model context	MCP redaction; secret values are never returned	Real

Team plan

Five workstreams, one owner each, joined by frozen contracts. The lead owns WS4, the contracts and the cut lines.

Workstream	Builds	Hands to others	Done by gate G1 (Oct 26)
WS1 Twin and order service	Twin Bicep (Container Apps, SQL, Key Vault, App Insights, identities); FastAPI order service with the seeded weakness and OTel; SQL schema and seed data; load generator; runbook v1 and seeded git history; CI pipeline (tests, scans, image to GHCR)	Order API, health endpoints, metric names, image tags	Order service runs green in the lab; removing access by hand breaks orders
WS2 Referee, fault and lab	Playwright journey and SQL check as a Container Apps Job; Ops Service (inject, restore, reset, deploy revision, kill); policy and conditional RBAC setup; `lab-probe.sh`; the `azd up` bootstrap; reset script	Referee result schema, Ops actions	Probe has run; one-command rebuild works; propagation delay measured; False Green reproduced by hand
WS3 MCP and evidence	Both MCP endpoints and all 12 tools: key auth, schema checks, redaction, spans. Also the AI Search index, the synthetic incident corpus, and GitHub tokens and rulesets	Tool schemas, evidence references	Tool schemas frozen; `/planner` returns fixture data

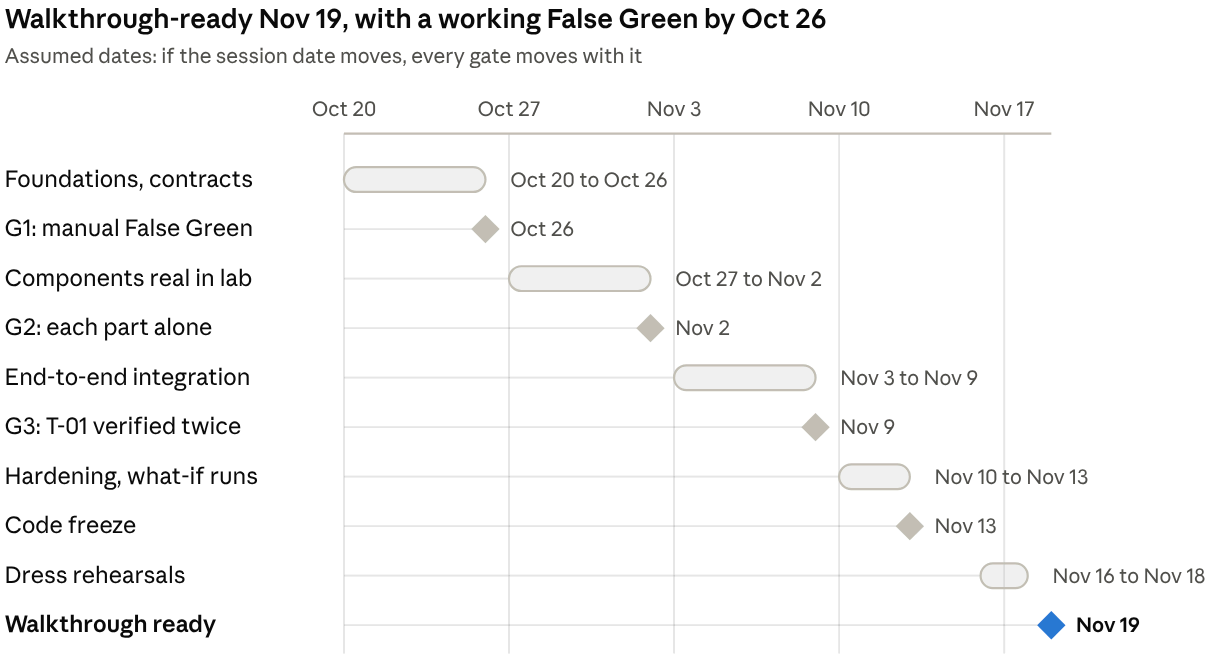
Workstream	Builds	Hands to others	Done by gate G1 (Oct 26)
WS4 Agents and workflow (lead)	`contracts` package; Foundry project, model, agents and Toolboxes as code; Agent Framework workflow, budgets, verdict and evidence packager; agent evaluation set	Drill state, event stream for the UI	Contracts frozen; workflow happy path runs on stubs
WS5 Control Center and story	Control Center UI: stage timeline, throughput and backlog, tool-call and denial panel, approval gate, scorecard, evidence view. Also the walkthrough run of show, deck and script updates, rehearsals and the fallback recording	The walkthrough	UI skeleton runs on stubbed events; list of deck corrections agreed

Owners. Name one owner per workstream at kickoff. WS2 suits whoever knows Playwright and Azure RBAC best. WS4 suits whoever has built Foundry agents.

Working rhythm

- 15-minute daily standup.
- One integration run in the lab every day from week 2.
- Any contract change needs the lead's approval and a version bump.

Four-week schedule



Schedule · four build phases, three gates, freeze, rehearsals

Each gate is a pass or fail check. If a gate is missed, we apply the cut lines in the Scope section instead of moving the dates.

- **G1, manual False Green:** One command builds a fresh lab. The order service and journey run green. Fault and restore work, and the propagation delay is measured. The False Green is reproduced by hand, and the contracts are frozen.
- **G2, each part alone:** The MCP endpoints return real evidence. Each agent runs on its own against the lab. The referee job, the Ops Service and CI to GHCR all work. The Control Center shows live stages.
- **G3, T-01 verified twice:** T-01 reaches VERIFIED end to end, twice in a row from a clean lab. The T-02, T-03 and T-04 denials are visible in the UI.
- **Code freeze:** 10 consecutive T-01 runs each end in VERIFIED or a named escalation. T-08 and T-09 pass. The fallback recording and the pre-validated candidate are ready.
- **Walkthrough ready:** two full dress rehearsals of the run of show are done.

Live customer walkthrough

Plan for a 60-minute session: a 45-minute guided drill in Supervised mode, then Q&A with a menu of prepared what-if runs. The length is assumed until the session is booked.

Minutes	What the customer sees	What it proves
0–5	Healthy orders per minute, zero backlog, the isolated twin, and the mode set to Supervised	Recovery is measured in customer transactions, and the autonomy level is set before the drill starts
5–10	The Ops Service removes the app's Key Vault access. Orders drop to zero and the backlog grows, while the replicas still show Running	The fault is bounded, reversible, traced and confined to the twin
10–17	The Planner's MCP calls, live in the Control Center and the Foundry trace, followed by broad and narrow recovery options with evidence	Agents investigate through governed, read-only tools
17–22	The customer picks the broad option and Azure Policy denies it. The narrow option is then approved and access is restored	Policy, not the model, decides what runs
22–27	Dependency health turns green, but the referee's journey fails, SQL shows no order, and the verdict reads FALSE_GREEN	Green infrastructure is not the same as business recovery
27–35	The Planner revises its view and writes the brief. The Engineer creates the branch, patch, test and runbook diff, and CI runs	The drill turns into a reviewable engineering fix
35–40	The candidate revision deploys, the same journey passes, the backlog drains, and the verdict reads VERIFIED	Evidence, not the agent, declares recovery
40–45	Scorecard, evidence package, the unmerged pull request, and residual risks	One drill produces both proof and a lasting improvement

The customer clicks the approval decision themselves. That gives the decision-latency number a real human behind it.

What-if menu for Q&A (each is a prepared run of a few minutes)

- A call to an Engineer tool made with Planner credentials is refused (T-03).
- An edit to the golden journey made with Engineer credentials is refused (T-04).
- A planted instruction in an incident record is ignored and has no tool effect (T-08).
- The kill switch is pressed mid-drill, the fault is reversed and the twin is reset (T-09).
- An Autonomous Twin re-run of the whole loop without pauses, left running in the background during Q&A.

The T-03 and T-04 denials come from scripted calls, so they do not depend on coaxing a model into misbehaving.

Before every session

- [ ] Launch the lab at least 90 minutes ahead, and confirm its remaining time covers the session
- [ ] Run the bootstrap, then the T-00 baseline
- [ ] Run one full T-01 rehearsal, then reset
- [ ] Check model quota, and confirm the `candidate/` branch namespace is empty
- [ ] Open tabs: Control Center, Foundry traces, App Insights, GitHub
- [ ] Have the fallback recording and the pre-validated candidate ready, both labelled as fallbacks

Risks

The two risks most likely to hurt are the lab's permissions and the lab being deallocated mid-session. Both are handled in week 1, before any agent work depends on them.

Risk	Likelihood	Impact	Mitigation	Owner
Lab is deallocated mid-build or mid-session	High	High	One-command rebuild; nothing lives only in the lab; launch early; confirm the lab duration	WS2
Lab blocks role or policy assignments	Medium	High	Day-1 probe; Path B; ask the organizers for Owner on the lab resource group	WS2

Risk	Likelihood	Impact	Mitigation	Owner
GPT-5.x not offered, or no quota, in the lab region	Medium	High	Probe on day 1; one shared deployment; tight token budgets; a smaller model tier as fallback	WS4
Access-change propagation (up to 30 minutes) stalls the story	Medium	Medium	Poll probes instead of waiting on timers; show the wait in the UI; network-rule fault as a fallback	WS2
Agent misses the hypothesis or writes a failing patch	Medium	Medium	Small curated evidence; structured outputs; a 10-run evaluation set before code freeze; two attempts; a pre-validated candidate, labelled as fallback	WS4
Scope creep from the deck	Medium	Medium	Cut lines in the Scope section; the lead approves any addition	WS4
A SKU or region policy blocks a service	Medium	Medium	Probe each service; use serverless SQL and the Basic or Free tier of AI Search	WS1
App registrations blocked, so no Easy Auth	Medium	Low	MCP already uses key auth; a passcode protects the approval gate	WS3
GitHub org rules or lab network egress block CI or MCP	Low	High	Confirm the org in week 1; the probe tests egress; tokens kept in Key Vault	WS3
Flaky browser journey	Low	High	Stable test IDs, one browser, explicit waits, pinned image	WS2
A locked immutability policy blocks lab cleanup	Low	Low	Keep retention unlocked and short in the PoC	WS1

Deck, script and open questions

Update the deck and script now so the walkthrough, the build and the submission all tell the same story.

Corrections

- Replace "Azure OpenAI GPT-4o" with the deployed GPT-5.x model, because gpt-4o was retired on Oct 1, 2026.
- "Azure AI Agent Service" is now Microsoft Foundry Agent Service. Add Microsoft Agent Framework and Foundry Toolboxes.
- Technology stack: drop AKS, Logic Apps, App Service, Power Apps and Azure Recovery Services, or mark them as roadmap. Add Container Apps, GitHub Actions and Playwright.
- Autonomy: the deck says human-in-the-loop, while the blueprint leads with Autonomous Twin. Use one line for both: "Policy-selectable: Supervised by default, Autonomous Twin inside the disposable twin, production always human-approved."
- Script slide 4 and storyboard scene 3 say the Planner restores access. In this design the Planner recommends and the workflow restores after approval, so fix the wording.
- Regenerate both architecture images. They contain garbled labels ("Intaramelmits", "Agent Rasing Management", "Root Couse Curse Reasoning", "Immutation of Duties", "CONTRCOPE"), and the data-flow image shows "Candidate Code Fix (04)" twice.
- Slide 6 says "Livefile Fusion Components". It should read LiveFire.
- Timing: the script runs 3 minutes, the storyboard 4 and the blueprint 7. Keep the 3-minute script for any recorded submission, and use the run of show above for the live session.

Open questions

- [ ] Exact walkthrough date and session length (the plan assumes Nov 19 and 60 minutes)
- [ ] Lab facts: duration per launch, launches per person, region, role, and whether a service principal is provided
- [ ] Which GitHub org hosts the repo, and whether it can be public (free CodeQL, GHCR pulls without credentials)
- [ ] Customer audience (CISO, engineering or both), which changes what the walkthrough emphasizes
- [ ] Whether the organizers can grant Owner on the lab resource group

Sources

- CloudLabs guided labs user guide
- What's new in Microsoft Foundry, July and August 2026 (Hosted Agents and Toolboxes generally available)
- What's new in Microsoft Foundry, Build 2026
- Microsoft Build 2026 partner summary (Agent Framework 1.0 generally available)
- Azure OpenAI gpt-4o retirement, October 2026
- Azure RBAC propagation latency, Microsoft Q&A
- Delegate role assignment management with conditions, Microsoft Learn